Отлично. Я считаю, что именно **Том 1А** станет самым важным документом всего проекта. Именно его обычно не хватает большинству самописных информационных систем. Из-за отсутствия единых правил через несколько месяцев разработки начинается хаос: разные названия таблиц, разные принципы создания карточек, несогласованные поля и дублирование данных.

Поэтому я предлагаю сделать **не просто методологию**, а **единый стандарт разработки TCMS**. Все последующие тома будут обязаны соответствовать этому документу.

ТОМ 1А

Методология проектирования информационной системы TCMS (Technical Control Management System)

Версия: 1.0
Статус: Базовый архитектурный стандарт проекта

---

1. Назначение документа

Настоящий документ определяет единые правила проектирования информационной системы TCMS.

Документ обязателен для всех модулей системы.

Любая новая таблица, форма, карточка, API, отчет или модуль должны соответствовать требованиям настоящего стандарта.

---

2. Основной принцип

TCMS проектируется **не вокруг таблиц**, а вокруг **бизнес-сущностей**.

Например, пользователь не работает с таблицей `Employees`.

Он работает с **Карточкой сотрудника**.

А таблица является только способом хранения данных.

Следовательно:

> **Карточка является основной единицей системы.**

---

3. Архитектурная философия

TCMS основана на четырех принципах.

Принцип №1

Одна информация — одно место хранения.

Например

ФИО сотрудника хранится только один раз.

Во всех остальных таблицах используется только ссылка (ID).

Никогда не допускается копирование текста.

Принцип №2

Все связи строятся через идентификаторы.

Никогда не используются текстовые связи.

Правильно

```
EmployeeID = EMP000245
```

Неправильно

```
Иванов Иван Иванович
```

Принцип №3

Карточка важнее таблицы.

Каждая бизнес-сущность обязана иметь карточку.

Принцип №4

История важнее текущего состояния.

Никогда не удаляются сведения.

Все изменения сохраняются.

---

4. Архитектурная модель

Вся система разделяется на четыре слоя.

```
Пользователь

↓

Интерфейс

↓

Бизнес-логика

↓

PostgreSQL
```

Ни один слой не может обращаться к другому напрямую, минуя бизнес-логику.

---

5. Стандарты проектирования базы данных

Все таблицы разделяются на четыре категории.

5.1 Справочники (Master Data)

Хранят постоянные данные.

Например

```
Employees

Projects

Objects

Contractors

Stages
```

```
Positions

Departments

Materials

Normatives
```

5.2 Рабочие таблицы (Operational Data)

Создаются ежедневно.

Например

```
Inspections

Remarks

Acts

LaboratoryTests

WorkLogs

Photos

Overtime

Visits
```

5.3 История

Хранят изменения.

Например

```
HistoryEmployee

HistorySalary

HistoryProject

HistoryObject

HistoryReport
```

5.4 Системные таблицы

```
Users

Roles

Permissions

AuditLog

SystemSettings

Templates
```

---

6. Правила именования

Таблицы

Используются только английские имена.

Правильно

```
Employees

Projects

Remarks

Inspections
```

Неправильно

```
Сотрудники

Таблица1

Employee_Data
```

Поля

Используется PascalCase.

```
EmployeeID

ProjectID

CreatedDate

CreatedBy

ModifiedDate

StatusID
```

Первичный ключ

Во всех таблицах

```
ID
```

тип

```
UUID
```

или

```
BIGINT
```

UUID предпочтителен для распределенной системы.

---

7. Стандарты карточек

Любая карточка состоит из одинаковых блоков.

```
Основная информация

↓

Связанные данные

↓

Документы

↓
```

```
Фото

↓

История

↓

Комментарии

↓

Журнал действий
```

Все карточки имеют одинаковую структуру.

Пользователь никогда не изучает новый интерфейс.

---

8. Обязательные поля каждой таблицы

Каждая таблица обязана содержать минимум:

```
ID

CreatedDate

CreatedBy

ModifiedDate

ModifiedBy

IsActive

Version

Comment
```

Это единый стандарт проекта.

---

9. Версионирование

Любая запись имеет номер версии.

Например

```
Версия 1

↓

Версия 2

↓

Версия 3
```

История не удаляется.

---

10. Архивирование

Удаление запрещено.

Вместо удаления запись получает статус

```
Archived
```

или

```
Inactive
```

Это сохраняет ссылки и историю.

---

11. Журнал изменений

Любое действие записывается.

Например

```
Кто

Когда

Что изменил

С какого значения

На какое значение

IP
```

```
Устройство

Комментарий
```

---

12. Правила создания форм

Каждая форма состоит из пяти зон.

```
Заголовок

↓

Панель действий

↓

Основная информация

↓

Связанные данные

↓

История
```

Все формы имеют одинаковый внешний вид.

---

13. Правила проектирования интерфейса

Интерфейс должен быть максимально простым.

Основные принципы.

Минимум цветов.

Минимум окон.

Максимум карточек.

Минимум ручного ввода.

Максимум автоматического заполнения.

---

14. Стандарты справочников

Каждый справочник обязан поддерживать:

✔ поиск

✔ фильтр

✔ архив

✔ историю

✔ импорт Excel

✔ экспорт Excel

✔ журнал изменений

---

15. Стандарты импорта

Любой импорт проходит одинаковые стадии.

```
Выбор файла

↓

Определение шаблона

↓

Нормализация

↓

Проверка

↓

Сверка

↓

Подтверждение

↓

Запись
```

```
↓

Журнал
```

---

16. Стандарты отчетов

Любой отчет строится одинаково.

```
Источник

↓

Поля

↓

Фильтр

↓

Группировка

↓

Расчеты

↓

Просмотр

↓

Excel

↓

PDF
```

---

17. Стандарты безопасности

Каждый пользователь имеет:

```
Роль

Права
```

```
Организацию

Подразделение
```

Каждое действие проверяется.

---

18. Производительность

Никогда не загружать всю таблицу.

Всегда используется:

- поиск;
- фильтрация;
- постраничный вывод (пагинация);
- серверная сортировка.

Это позволит системе работать даже при миллионах записей.

---

19. Методология развития

Любой новый модуль обязан отвечать на пять вопросов:

1. Какую бизнес-задачу он решает?
2. Какие новые сущности появляются?
3. Какие существующие сущности он использует?
4. Какие отчеты формирует?
5. Какие права доступа требуются?

Если хотя бы на один вопрос нет ответа, модуль не принимается в разработку.

---

20. Принцип "Конструктор, а не программист"

Одной из стратегических целей TCMS является **минимизация зависимости от программистов**.

Поэтому каждый новый функциональный модуль должен, по возможности, реализовываться через универсальные механизмы системы, а не через уникальный программный код.

Это означает, что в системе должны существовать конструкторы:

- конструктор справочников;

- конструктор карточек;
- конструктор форм;
- конструктор документов;
- конструктор отчетов;
- конструктор маршрутов согласования;
- конструктор уведомлений;
- конструктор вычислений (формул);
- конструктор ролей и прав доступа.

Например, если потребуется создать новый документ (акт, журнал, ведомость), администратор должен иметь возможность собрать его из существующих полей и правил без изменения исходного кода системы.

---

21. Архитектурный принцип "Борей 2.0"

В классическом Microsoft Access "Борей" основными элементами были:

- таблицы;
- запросы;
- формы;
- отчеты.

В TCMS эта философия расширяется:

Архитектура "Борей"	Архитектура TCMS
Таблицы	Бизнес-сущности и карточки
Запросы	Конструктор запросов и аналитики
Формы	Веб-карточки и мастера действий
Отчеты	Конструктор отчетов, Dashboard, Excel, PDF
Макросы	Серверная бизнес-логика Python
Модули VBA	Сервисы FastAPI и фоновые задачи

Таким образом, TCMS сохраняет простоту и понятность "Борея", но получает современную масштабируемую архитектуру, пригодную для многолетнего развития.

Заключение

Этот документ должен стать **Конституцией проекта**. Любое решение — от создания новой таблицы до разработки нового модуля — должно проверяться на соответствие этим правилам.

Моя рекомендация

После Тома 1 и Тома 1А **не переходить сразу к проектированию таблиц PostgreSQL**. Для системы такого масштаба целесообразно сначала подготовить **Том 1Б — "Бизнес-архитектура TCMS"**, где будут описаны все бизнес-сущности (Сотрудник, Объект, Проект, Замечание, Инспекция, АОСР, Документ, Испытание и т.д.), их жизненный цикл, взаимосвязи и основные бизнес-процессы. Это позволит построить структуру PostgreSQL (Том 2) не по отдельным таблицам, а по реальным процессам технического надзора, что сделает систему логичной, расширяемой и устойчивой к изменениям.